Nano UAV Mapping and Exploration Application

Trent Rossos
Student ID: 1008073914
Supervisor: Prof. Beno Benhabib
Co-Supervisor: Owais Mateen Abdul Hamid
April 2026

B.A.Sc. Thesis

Division of Engineering Science
UNIVERSITY OF TORONTO

Abstract

This report presents the design and implementation of a fully onboard mapping and autonomous exploration system for the Bitcraze Crazyflie 2.1 nano-UAV, developed as a foundational step toward decentralized multi-agent SLAM. The system operates without any external positioning infrastructure or offboard computation, running entirely on the GAP8 RISC-V processor provided by the AI Deck 1.1 expansion board and the STM32 flight controller.

A 2D log-odds occupancy grid is built in real time using Bresenham ray casting applied to sparse time-of-flight sensor data from the Multi-ranger deck. A frontier-based exploration algorithm runs onboard the GAP8 to autonomously identify unexplored regions and generate navigation waypoints, which are transmitted back to the STM32 flight controller over the Crazyflie Packet eXchange (CPX) protocol for execution. The occupancy grid is streamed incrementally to a ground station over WiFi using a delta encoding scheme that reduces bandwidth by approximately 95% compared to full map transmission, enabling real-time visualization in RViz2 via a custom ROS2 node.

The work is directly motivated by the NanoSLAM [7] and C-SLAM [8] frameworks developed at ETH Zurich, and adopts their core mapping representation while deliberately omitting scan matching to focus on the navigation pipeline as a foundation for future localization work. Hardware and software limitations of the Crazyflie platform are discussed in detail, and a path forward toward custom drone hardware and full Iterative Closest Point (ICP) based SLAM is proposed.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Prof. Beno Benhabib and Owais Mateen Abdul Hamid, for their guidance, support, and encouragement throughout this project. Their expertise and feedback were invaluable in shaping both the direction and quality of this work. I am also grateful to the broader Bitcraze and open-source robotics communities whose documentation, example code, and community contributions made this research possible. Finally, I would like to thank my family and friends for their patience and support throughout my undergraduate studies.

Contents

1 Introduction **1**
1.1 Context and Motivation . . . 1
1.2 Research Objectives . . . 1

2 Relevant Research **1**
2.1 Motivation . . . 1
2.2 Prior and Current Approaches . . . 1
2.3 Core Contributions of the Nano-SLAM Framework . . . 2
2.3.1 Local Mapping and Pose-Graph SLAM . . . 2
2.3.2 Intra-Drone Loop Closures via Scan-Matching . . . 3
2.4 Core Contributions of the C-SLAM Framework . . . 3
2.4.1 Inter-Drone Loop Closures and Map Alignment . . . 4
2.4.2 Communication Protocol and Data Management . . . 4
2.4.3 Performance and Limitations . . . 5
2.5 Proposed Developments . . . 5

3 The Crazyflie Platform **5**
3.1 Crazyflie 2.1 . . . 5
3.2 Perception Expansion Decks . . . 7
3.2.1 Flow Deck v2 . . . 7
3.2.2 Multi-ranger Deck . . . 8
3.3 AI Deck 1.1 . . . 8

4 Firmware Development **9**
4.1 Development Environment Configuration . . . 9
4.2 Developing STM32 Software . . . 9
4.2.1 Inital Testing . . . 9
4.3 Developing GAP8 Software . . . 10
4.4 ESP32 Firmware . . . 11
4.5 Crazyflie Packet eXchange (CPX) . . . 11

5 Onboard Mapping and Navigation Implementation **12**
5.1 Utilizing CPX for communication . . . 12
5.2 STM32 Firmware . . . 13
5.2.1 Processing ToF Data . . . 13
5.2.2 Waypoint Processing . . . 14
5.3 GAP8 Firmware . . . 15
5.3.1 Occupancy Grid Mapping . . . 15
5.3.2 Bresenham's Ray Tracing . . . 16
5.4 Frontier-Based Exploration . . . 17
5.5 Host Software and ROS2 Integration . . . 19

5.6 Repository Layout . . . 21
5.7 Building and Deploying the Code . . . 22
5.7.1 GAP8 . . . 22
5.7.2 STM32 . . . 22
5.7.3 ESP32 . . . 22
5.7.4 Client Side ROS2 Vizualizer . . . 23
5.7.5 Running the Mapping and Exploration . . . 23
5.8 Evaluation and Results . . . 23

6 Hardware Limitations and Challenges **24**
6.1 Flight Dynamics . . . 24
6.2 Battery Life . . . 24
6.3 Sensor Sparsity . . . 25
6.4 Compute Headroom . . . 25
6.5 Practical Applications . . . 25
6.6 Software Ecosystem . . . 25

7 Conclusion **26**

A STM32 Developed Code **30**

B AI Deck Developed Code **39**

C ROS2 Client Code **47**

List of Figures

1 The Crazyflie 2.1 drone. 6
2 The `cfclient` interface, showing the drone telemetry and flight control interface. 7
3 Preliminary point cloud visualization of the ToF data from the Multi-ranger deck, developed using `cflib`. 10
4 Directory structure of the AI Deck GAP8 mapping application. 10
5 Flow chart of System architecture outlining how the different MCU Communicate 12
6 Diagram illustrating the concept of frontier cells in an occupancy grid. The red cells represent frontier cells, which are unknown cells adjacent to free space. . . . 18
7 Flowchart outlining Frontier Based Exploration 19
8 RViz visualization showing the occupancy grid map and drone pose. The map is rendered in grayscale, with occupied areas appearing darker and free space lighter. The drone's current pose is indicated by a axis marker aligned with its yaw. . . . 20
9 Repository structure. Files marked with * are developed as part of this work. . . 21

1 Introduction

1.1 Context and Motivation

Multi-UAV systems have demonstrated significant potential in search and rescue, surveillance, and environmental monitoring applications. Previous CIMLab work validated iso-probability search algorithms using the Crazyflie platform from bitcraze swarms with centralized ground control [6]. However, centralized architectures introduce single points of failure and communication bottlenecks that limit scalability and robustness. Fully decentralized swarms, where UAVs operate autonomously without ground station dependency, remain challenging due to onboard computational and sensing constraints. Recent advances in edge computing and miniaturized sensors enable decentralized coordination on nano-quadcopters. ETH Zurich recently demonstrated fully onboard SLAM and distributed mapping with nano-UAV swarms, achieving collaborative environment mapping without external infrastructure [8]. This work builds upon these advances by developing onboard capabilities for inter-drone recognition and independent navigation, enabling coordinated swarm behavior without centralized control.

1.2 Research Objectives

The objective of my thesis is to implement a fully onboard mapping and exploration system for the Crazyflie 2.1 nano-UAV, no previous knowledge of room conditions or initial positions. This objective can be broken into the following milestones. (1) Map test course using Crazyflie with support from hub station. (2) Develop communication for inter board communication. (3) Generate a map of the test course using manual flight control. (4) Autonomously map and explore the test course without knowledge of the initial starting position or environment.

2 Relevant Research

2.1 Motivation

Accurate SLAM with fully onboard computation is routine for midsize and high end robotic platforms equipped with powerful CPUs or GPUs and rich sensor suites. However, translating these capabilities to swarms of nano, low cost robots is substantially more challenging because each agent is severely constrained in weight, power, memory, sensing, and communication bandwidth.

Most existing UAV SLAM systems assume either: (1) a powerful centralized ground station that performs global optimization and map fusion using data offloaded from the robots, or (2) relatively capable robots that can run heavy visual or LiDAR SLAM back-ends and maintain dense maps individually. Both strategies are unsuitable for swarms of centimeter-scale drones, where each robot has on the order of 1–2 MB of memory and a power budget in the order of hundreds of milliwatts for all perception and computation [5].

2.2 Prior and Current Approaches

This thesis will build on 'Nanoslam: Enabling fully onboard slam for tiny robots' by Niculescu et al. [7], which presents an innovative way to implement onboard slam for nano-UAVs. Niculescu

et al were able to achieve 4.5 cm mapping accuracy using only four VL53L5CX time of flight (ToF) sensors (front/back/left/right), GAP9 parallel processor, and graph based SLAM with ICP (iterative closest point) running in under 250 ms at 88 mW on a 44 g platform. Its core innovations include aggregating sparse ToF 'scan frames' into full 360° scans (via controlled drone rotation), hierarchical pose-graph optimization to fit <500 kB RAM, and parallel ICP implementation (55 ms per match).

The related collaborative SLAM framework was developed in the article "Ultra-lightweight collaborative slam for robot swarms" by Niculescu et al. [8]. This paper extends the single-drone techniques to swarms, adding inter-drone loop closures (ICP between overlapping scans from different UAVs) and ID based pose anchoring to maintain global consistency without centralized fusion. Since the C-SLAM paper is a progression of the original NanoSLAM paper [7], it imposes the same nano-UAV constraints: 1–2 MB total memory, hundreds of mW compute budget, centimeter-scale ToF sensors, and no external infrastructure. Additionally, these two papers share the same scan representation, ICP front-end, and lightweight PGO back-end, but C-SLAM introduces token-passing communication protocols and distributed optimization for scalability to hundreds of agents.

This project positions NanoSLAM as the single-drone foundation while targeting C-SLAM's swarm coordination with a key innovation of eliminating their shared assumption of known initial poses, enabling truly autonomous deployment in unknown environments using off the shelf Bitcraze hardware (AI-deck GAP8, Multi-Ranger deck).

2.3 Core Contributions of the Nano-SLAM Framework

2.3.1 Local Mapping and Pose-Graph SLAM

At the heart of the NanoSLAM method is a graph-based SLAM formulation [7] that enables efficient memory usage. The drone represents its trajectory as a pose graph where nodes correspond to 2D poses $x_k = (x_k, y_k, \psi_k)$ in the world frame at discrete times, and edges encode relative motion constraints between poses. The graph comprises two edge types: (1) odometry edges $z_{i,i+1}$ incorporating motion information between consecutive poses (computed via Eqs. 2-3), and (2) loop closure (LC) edges embodying relative measurements derived by ICP when the robot revisits previously observed regions.

Instead of building dense maps from rich RGB-D or LiDAR data, NanoSLAM uses five ToF sensors producing sparse range measurements in four directions (front, back, left, right, up). At each pose, the system records a "scan frame" by projecting 8-zone measurements to 2D world coordinates (middle 4 rows median-filtered to 32 values/sensor). These scan frames are aggregated into full 360° "scans". During data acquisition the drone rotates 45° to achieve complete angular coverage, yielding 640-point scans stored compactly for memory-constrained platforms [7].

Another integral part of the SLAM method outlined in the NanoSLAM paper is Pose Graph Optimization (PGO). Given the inherent noise present in the drones odometry measurements it is important to globally correct the drones current position. PGO estimates optimal poses $X^* = \{x_0, \ldots, x_{N-1}\}$ minimizing inconsistency between measured edges $z_{ij} = (z_x, z_y, z_\psi)$ and

predictions $\hat{z}_{ij}(x_i, x_j)$:

$$e_{ij} = z_{ij} - \hat{z}_{ij}(x_i, x_j), \quad X^* = \arg\min_X \sum_{i,j} e_{ij}^T \Omega e_{ij} \tag{1}$$

where Ω weights LC edges higher ($20\boldsymbol{I}$) than odometry ($\boldsymbol{I}$) due to ICP precision. Odometry edges use:

$$\begin{pmatrix} z_x \\ z_y \end{pmatrix} = R^{-\psi_i} \begin{pmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \end{pmatrix}, \tag{2}$$

$$z_\psi = \psi_{i+1} - \psi_i \tag{3}$$

with $x_0 = (0, 0, 0)^T$ fixed post-takeoff.

The non-linear problem is linearized using first order Taylor expansion around current estimate x^*, yielding a sparse system $H\Delta x = -b$, which can be solved iteratively. NanoSLAM stores H (3N×3N) in CSR sparse format exploiting bandwidth structure (odometry near-diagonal, LC sparse). Cholesky-Crout decomposition is used to allow for more space efficient matrix calculations.

This sparse PGO implementation was able to store 440 poses within 1.5MB RAM, with hierarchical extension for trajectories exceeding direct limits [7].

2.3.2 Intra-Drone Loop Closures via Scan-Matching

When a drone revisits an area it has previously scanned, the system performs ICP between the current scan and candidate past scans whose poses are spatially close. This scan-matching step produces a rigid-body transform that aligns the two scans in the world frame. The resulting transform is then converted into a loop-closure edge between the corresponding poses in the pose graph. [7]

By incorporating these intra-drone loop closure edges into the pose graph, the system corrects accumulated drift from the odometry edges. PGO minimizes the discrepancy between measured and predicted edge transformations (for both odometry and loop closures) over all poses, yielding a globally consistent trajectory. Critically, the optimization is deliberately kept lightweight so that it can run onboard in real time within the limited memory and power constraints of each nano-UAV [8].

2.4 Core Contributions of the C-SLAM Framework

The paper "Ultra-lightweight collaborative slam for robot swarms" by Niculescu et al. utilized many of the same algorithms and methods described in the NanoSLAM paper [8], however the novelty comes from the added collaboration between multiple drones in a swarm. These innovations have two crucial aspects, an inter-drone communication piperine and collaborative map optimization.

Firstly, the paper proposes a communication and state-estimation pipeline that scales efficiently with swarm size. Rather than streaming raw sensor data, each drone transmits compact

scan and pose graph summaries sufficient to form loop-closure constraints. This minimizes bandwidth usage and allows hundreds of robots to cooperate over standard wireless technologies such as ultra-wideband (UWB), Bluetooth, or WiFi [8].

Secondly, they propose a distributed swarm coordination and optimization scheme that reduces mapping latency and maintains global consistency. Each agent runs its own local SLAM backend, while lightweight inter-drone loop closure constraints and a hierarchical anchoring scheme align all local maps into a shared global frame. The resulting system is both distributed and low latency, enabling fast coverage of unknown environments by large swarms without centralized optimization [8].

2.4.1 Inter-Drone Loop Closures and Map Alignment

The collaborative aspect of the framework arises from inter-drone loop closures. When two drones observe overlapping regions of the environment, their scans contain compatible geometric structures. The system leverages this overlap to compute a rigid transform between scans originating from different robots using the same ICP procedure as for intra-drone loop closures. This transform defines an inter-drone loop-closure constraint that connects their previously independent pose graphs. [8]

To keep the optimization distributed and scalable, the authors introduce an ID-based anchoring strategy. Each drone maintains its own pose graph anchored at its first pose x_0, which is never modified by PGO. When a higher-ID drone detects an inter-drone loop closure with a lower-ID drone, it treats the lower-ID poses as fixed anchors and encodes the inter-drone constraint as an additional edge to its own x_0, rather than introducing explicit external nodes. This trick preserves the influence of the other robot's map while avoiding quadratic growth in the number of nodes and ensuring that the lower-ID maps remain stable reference frames. [8]

Moreover, when lower-ID drones later update their poses via PGO, they broadcast only updated pose values, not entire scans. Higher-ID drones update their stored inter-drone constraints by composing these pose updates with the original ICP transform. This incremental update mechanism preserves consistency across the swarm while keeping communication and computation overhead low. [8]

2.4.2 Communication Protocol and Data Management

The framework is accompanied by a carefully designed communication protocol that runs in multiple stages under a token passing scheme. Only the token holder executes computationally expensive tasks such as ICP and PGO, while other drones stay in a low power listening mode. During its active phase, a drone performs UWB ranging with higher ID robots, processes external scans to derive inter-drone constraints, broadcasts newly acquired scans and internal loop closures, optionally runs pose-graph optimization, and finally disseminates updated scan poses involved in loop closures. [8]

On the data structure side, each drone stores augmented poses, internal scan indices, an external scan buffer, and an external constraints list. The external buffer temporarily holds scans received from other drones until they are matched with local scans; the constraints list stores inter-drone edges along with their associated ICP transforms, allowing fast updates when

external poses change. All these structures are implemented with tight memory budgets (on the order of a few hundred kilobytes) and are processed by a low-power parallel processor (e.g., GAP9), while a microcontroller (e.g., STM32) handles sensing, navigation, and communication. [8]

2.4.3 Performance and Limitations

The proposed C-SLAM framework achieves mapping accuracy on the order of tens of centimeters, comparable to state-of-the-art systems that rely on much more powerful hardware. At the same time, the total sensing and computation power budget remains below approximately one watt per robot, and the system operates on platforms with around 1.5 MB of memory, which is two orders of magnitude smaller than typical high-end SLAM deployments. [8]

The C-SLAM communication pipeline supports large swarms, with data traffic that remains bounded as the number of agents increases. Because each drone only broadcasts compact scan and pose information and uses a token-based schedule for heavy processing, the framework can coordinate hundreds of agents using standard wireless technologies. This makes the approach attractive for applications such as large scale exploration, search and rescue, or industrial inspection with low-cost aerial swarms. [8]

2.5 Proposed Developments

Both papers propose solutions that require a known global initial position with exact certainty. This assumption is not valid in most application of this technology, therefore the key innovation my project will explore is being able to map without initial pose knowledge. Furthermore, I will be completing this task using only off the shelf components, thus creating a reproducible product others can implement.

3 The Crazyflie Platform

3.1 Crazyflie 2.1

The Crazyflie 2.1 from Bitcraze was chosen for this project due to being a continuation of the hardware developed and used in NanoSLAM [7]. The custom PCBs they developed in the paper inspired the design for what was eventually turned into a suit of products compatible with the Crazyflie 2.1 platform. This made it an obvious choice, due to one of my goals being to replicated the results of NANOSlam.

The Crazyflie 2.1 drone is a micro UAV weighing in at only 29g and fitting in the palm of your hand [3]. It is equipped with a STM32F405 microcontroller, running at 168MHz, and having 192kb SRAM. The drone is powered by a 100mA one cell LiPo battery and has a 3 axis accelerometer and a pressure sensor that are used as input for the onboard extended Kalman filter.

The Crazyflie 2.1 firmware is written in C employing an RTOS scheduler and is open source. Additionally, Bitcraze created a python library `cflib`, that allows user to easily create python scripts that can interface with the drone over radio using their CrazyRadio [4]. Furthermore,

Figure 1: The Crazyflie 2.1 drone.

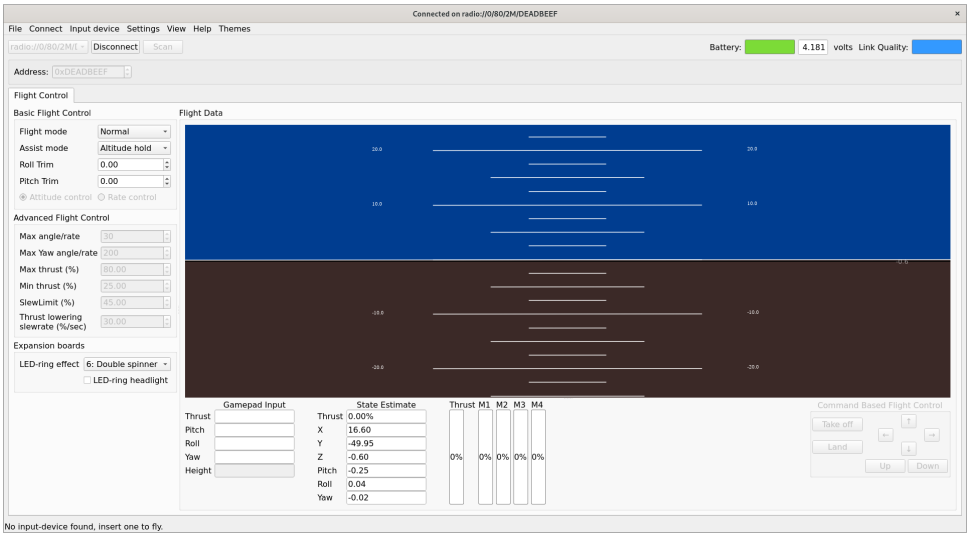

Figure 2: The `cfclient` interface, showing the drone telemetry and flight control interface.

Bitcraze have develped a client side flight control / monitor, and logging interface `cfclient` allowing for easy control, debugging, and development, as shown in Figure 2. The drones interface with the client over 2.4GHz radio frequency, where the client machine is using the Crazyradio 2.0.

3.2 Perception Expansion Decks

The crazyflie 2.1 platform was designed to be modular, different boards can be used to customize the drone's capabilities. For the scope of this thesis, three expansion decks were utilized:

- **Flow Deck v2:** Uses the PMW3901 optical flow sensor for velocity estimation.
- **Multi-ranger Deck:** Employs 5× VL53L1x sensors for laser ranging in the body frame.
- **AI Deck 1.1:** Features the GAP8 SoC (8+1 RISC-V cores) and 64 MB of external RAM.

3.2.1 Flow Deck v2

The FLow Deck v2 includes a downward facing VL53L1x ToF sensor improving the altitude estimation and a PMW3901 optical flow sensor allowing the drone to measure its movement in relation to the ground. These additional sensing capabilities allow for much more accurate state estimation giving the user the ability to do position control, IE command the drone to move to specified poses in space. Since optical flow is camera based, the position estimation works best when flying over feature rich environments like checkerboards or textured surfaces. This deck is essential for increasing the accuracy of the drones state estimation, which is crucial for the mapping application, as the drone needs to know where it is in space in order to create an accurate map of the environment.

3.2.2 Multi-ranger Deck

The Multi-ranger deck contains five VL53L1x time-of-flight sensors oriented in the five cardinal directions: front, back, left, right, and up. Each sensor operates by emitting a laser pulse and measuring the time taken for the reflection to return, converting this into a single scalar distance measurement with a range of up to 4 meters [1]. This single scalar per sensor is an important distinction from more capable ToF solutions. The VL53L5CX sensors used in NanoSLAM, for example, use a single device to produce an 8×8 zone measurement matrix, yielding 64 distance values per sensor and up to 32 usable horizontal points per scan after filtering. The Multi-ranger deck by comparison produces only one distance value per sensor, giving a maximum of four usable horizontal points per scan after excluding the upward-facing sensor. This four-to-one disparity in spatial sampling is the root cause of the sparse occupancy grids observed during testing, where walls manifest as dotted lines rather than solid boundaries, and directly limits the feasibility of scan matching techniques such as ICP that require sufficient geometric overlap between consecutive scans to converge reliably.

3.3 AI Deck 1.1

Finally, the AI Deck 1.1 is being used due to the increased compute capabilities it facilitates. The AI deck is equipped with two MCUs, a RISC-V GAP8 processor, and a ESP32 for WiFi capabilities. The has an large 512 Mbit HyperFlash and 64 Mbit HyperRAM [2]. Testing with only the STM32 on the base Crazyflie revealed that the compute capabilities and the limited amount of RAM (192kb), are not sufficient for the mapping application. The RAM usage when running the default firmware can be seen in table 1. With the base firmware flashed, only 46kB of RAM available for the entire mapping architecture, this is not enough making the AI deck a nessisity. Additionally, the extra compute is helpful full increased computation speed and creates more headroom for future projects such as VSlam.

Additionally, the ESP32 on board provides the drone with WiFi capabilities allowing it to wireless communicate with client PCs and potentially other drones in a swarm configuration. The ESP32 can be configured to either be access point or connect to an existing network. This allows for a lot of flexibility in the communication architecture, as the drone can either create its own network or join an existing one. This is crucial for the mapping application, as it allows for the drone to stream the occupancy grid to a client PC in real time, allowing for visualization and debugging of the mapping process.

Memory Region	**Usage (%)**	**Used (kB)**	**Total (kB)**	**Free (kB)**
Flash (Program)	27%	277.1	1008.0	730.9
SRAM (System)	64%	82.1	128.0	45.9
CCM (Critical)	95%	60.7	64.0	3.3

Table 1: STM32 Memory Utilization and Allocation Breakdown (in kB)

4 Firmware Development

4.1 Development Environment Configuration

All the tooling and development tools for the Crazyflie ecosystem are intended to be run on a linux machine,although available resources made developing on Windows a more attractive option. So Windows Subsystem for Linux (WSL) was employed to bridge the gap between Windows-based productivity tools and the Linux-centric ROS2 and Bitcraze toolchains. However, this configuration presented significant technical hurdles regarding hardware passthrough. Since WSL and the Windows host share the same physical hardware, exposing the Crazyradio PA USB dongle to the WSL kernel required complex USBIP (USB over IP) configurations which proved unstable during high-bandwidth telemetry tasks. Consequently, the development environment was migrated to a native Ubuntu 20.04 LTS desktop. This transition eliminated the virtualization layer, providing direct access to the USB bus and display drivers, thereby streamlining the firmware compilation and RViz2 visualization processes.

Overall, it was found that while WSL offers a convenient bridge for Linux tools on Windows, the performance, stability, and ease of use made a native Linux environment the more reliable choice for this project.

4.2 Developing STM32 Software

The Crazyflie firmware is open source allowing for easy development of custom applications. The firmware uses FreeRTOS for task scheduling which allows for efficient management of multiple concurrent tasks. FreeRTOS uses semaphores and other locks to ensure there are no data races and no blocking processes, which are essential qualities in embedded systems.

The best way to build an application for the Crazyflie platform is to utilize the "App Layer", which is a set of functionalities and APIs that allow users to add custom code on top of the provided base firmware. Furthermore, the app layer supports out of the tree (OOT) builds, meaning the custom code development can be done separately of the firmware repository, allowing for easier code mergers and better code organizations. The firmware is designed to be built using GNU make, with K-Build to customize the specific build options.

4.2.1 Inital Testing

Preliminary testing was done using the `cflib` library to verify that the drone was able to receive data from the Multi-ranger deck and experiment with sensor FOV. A simple point cloud visualization was created using the data from the Multi-ranger deck, which can be seen in figure 3, where the resulting postcloud is displayed. This simple experiment was useful for understanding the data being received from the Multi-ranger deck and how to process it for use in the mapping application. This application did not do any of the mapping calculation onboard the drone, but it was a good first step in understanding the data and how to work with it. It also provided a good testbed for developing the firmware for the STM32, as it allowed for easy testing of the data processing and communication between the STM32 and the GAP8.

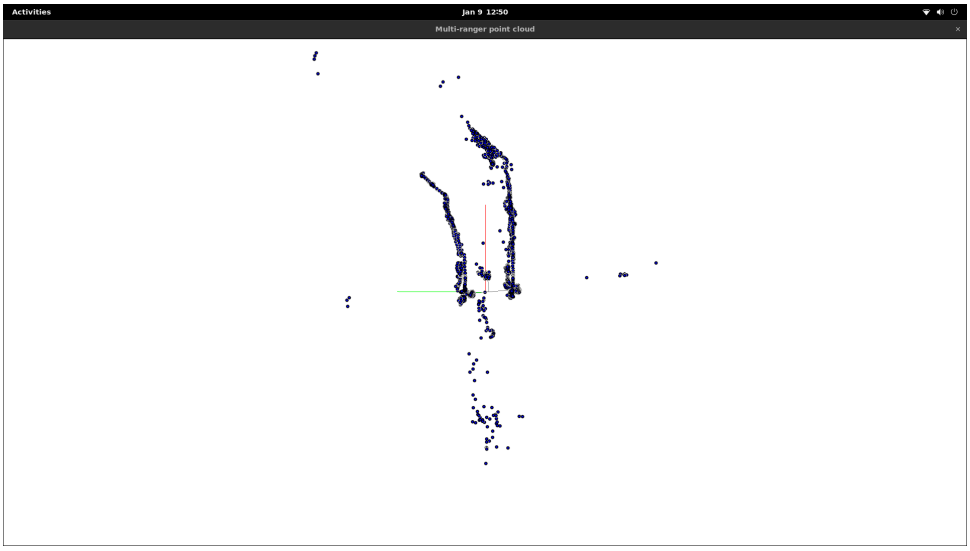

Figure 3: Preliminary point cloud visualization of the ToF data from the Multi-ranger deck, developed using `cflib`.

4.3 Developing GAP8 Software

Developing for the GAP8 on the Crazyflie requires building a co-processor application that must stay perfectly synchronized with the STM32 flight controller. Unlike the STM32, the GAP8 lacks internal flash memory and executes code directly from its 512 kB L2 RAM. To provide persistence across power cycles, the AI-deck utilizes an external HyperBus Flash chip. Upon startup, a resident bootloader on the GAP8 checks this external flash for a valid application image; if the hash computation succeeds, the bootloader copies the binary into L2 and begins execution [2].

Code development happens inside the /aideck-gap8-examples/examples/**Custom application**, as seen in figure 4. Similarly to the STM32, the GAP8 firmware is built upon FreeRTOS abstraction layer. Allowing for allocation of tasks based on priority, facilitating fast non blocking code.

```
aideck-gap8-examples/
├── docs/
├── examples/
│   ├── ai/
│   ├── Custom Application/
│   │   ├── BUILD/
│   │   └── Makefile
│   ├── image_processing/
│   └── other/
├── lib/
├── tools/
└── module.json
```

Figure 4: Directory structure of the AI Deck GAP8 mapping application.

4.4 ESP32 Firmware

The main function of the ESP32 is to serve as a WiFi bridge between the GAP8 and the client computer. It is able to receive messages from the GAP8 over SPI and then transmit them over WiFi to the client. The ESP32 is also responsible for receiving messages from the client and transmitting them back to the GAP8. The ESP32 acts as a middle man and all the CPX writes to WiFi are automatically sent to the ESP32, and all the CPX reads from WiFi are automatically read from the ESP32. The ESP32 is set up to be an access point, allowing the devices on the same network to connect to it and receive data being sent from the GAP8.

Initial testing using the stock Bitcraze video-streaming implementation revealed significant instability in the communication bridge. The ESP32, acting as the Access Point (AP) and SPI-to-WiFi bridge, frequently failed to process the high-volume data packets originating from the GAP8 processor. This resulted in critical packet loss and system crashes.

To resolve this, a community-developed firmware solution provided by Marijana Peti (@marijana23) from the Laboratory for Robotics and Intelligent Control Systems (LARICS) at the University of Zagreb was implemented. This workaround utilizes optimized forks for both the ESP32 firmware and the GAP8 streaming application, significantly improving buffer management and SPI synchronization [9].

The LARICS optimization replaces the standard, high-overhead communication stack with a lightweight UDP-based bridge that prioritize throughput over error correction. In the original Bitcraze firmware, the ESP32 often struggled with the "Stop-and-Wait" nature of TCP-like protocols, where a single lost packet would stall the entire SPI bus while waiting for a retransmission. The workaround implements an asynchronous UDP socket on the ESP32 that immediately broadcasts incoming SPI data from the GAP8 as it arrives. By utilizing Zero-Copy buffers and stripping away the transport layer handshaking, the system eliminates blocking. This allows the GAP8 to continuously send image data at the maximum hardware clock speed of the SPI interface, ensuring that the drone's navigation stack receives a consistent, telemetry stream without the latency spikes that previously caused system-wide instability and crashes.

4.5 Crazyflie Packet eXchange (CPX)

The Crazyflie Packet eXchange (CPX) serves as the high-level communication backbone of the system, enabling seamless data routing between the four primary nodes: the STM32 flight controller, the GAP8 AI processor, the ESP32 WiFi module, and the external host computer. Because these microcontrollers use different physical interfaces, specifically SPI between the STM32 and GAP8, and UART/WiFi between the GAP8 and host, CPX abstracts these layers into a unified routing protocol based on the source, destination, and function.

To maintain compatibility across the different CPU architectures (ARM Cortex-M4 on the STM32 and RISC-V on the GAP8), all data structures are defined using the `__attribute__(( packed))` qualifier. This prevents compilers from adding padding bytes, ensuring that any struct sent by the STM32 is byte-aligned perfectly for the GAP8's memory.

The STM32 and the GAP8 firmware have a slight difference in the way they handle packets.

The GAP8 allows for granular, per-function registration. Using `cpxEnableFunction(CPX_F_APP)`, the application layer registers interest in a specific ID. This allows for a clean, event-driven approach. The `cpxReceivePacketBlocking()` call is used to receive a packet. The task remains in a power-saving blocked state until a packet with the matching ID arrives, ignoring all other traffic. Due to the RTOS scheduler, this does not block other tasks from running, allowing for efficient multitasking on the GAP8.

The STM32 implementation is more rigid. Its internal router uses a hardcoded switch statement to decide which packets to keep. If a packet arrives with a Function ID not explicitly defined in the STM32's firmware (e.g., a custom `CPX_F_WAYPOINT ID`), the router silently drops the packet. In the default Bitcraze firmware, only a few IDs like `CPX_F_APP(5)` and `CPX_F_CONSOLE (100)` are permitted to pass into the application-level queues. To work around this, a manual multiplexing approach is implemented, which is described in more detail in section 5.1.

5 Onboard Mapping and Navigation Implementation

A fully embedded mapping and navigation algorithm was developed for the Crazyflie 2.1 equipped with the AI Deck 1.1, Flow deck v2, and the Multi-ranger deck. The system uses the STM32 onboard the Crazyflie to package ToF data from the multi-ranger deck and transmit it to the GAP8 for the more computationally expensive tasks of mapping and navigation. The GAP8 uses these ToF packets to build a map of the environment and compute optimized waypoints to efficiently explore the environment. All while the ESP32 relays the map data over WiFi to a client desktop running a ROS2 RViz visualization. The detailed system architecture can be seen in figure 5

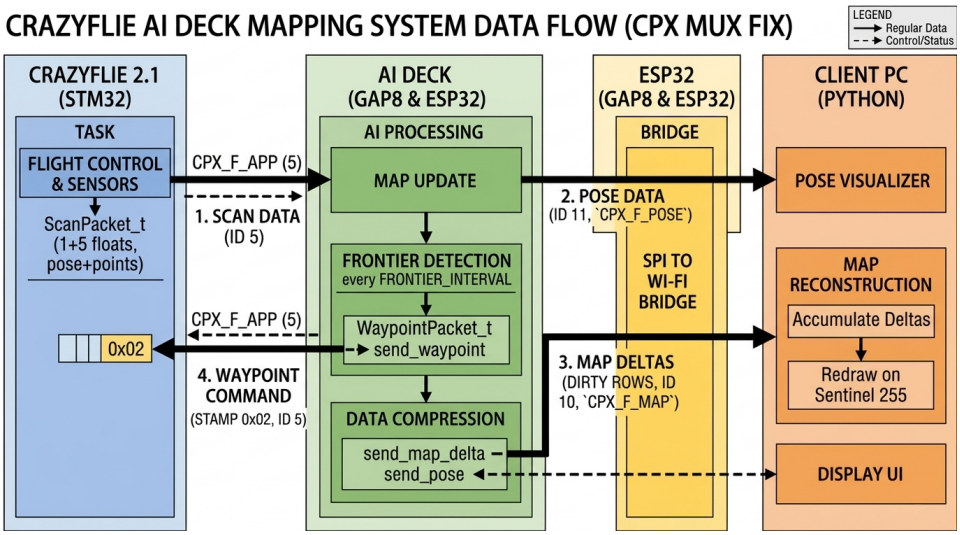

Figure 5: Flow chart of System architecture outlining how the different MCU Communicate

5.1 Utilizing CPX for communication

To address the architectural bottleneck of the STM32's static routing, the implementation reuses the existing `CPX_F_APP` ID for both incoming sensor scans and outgoing navigation waypoints. A

manual multiplexing layer is implemented where the first byte of the data payload is reserved as a type identifier. On the GAP8, outgoing waypoints are prepended with a specific tag (0x02), while the STM32 side implements a manual filter to intercept and deserialize these packets from its shared `mixedQueue`, ensuring that the autonomous navigation loop remains closed and responsive without modifying the core flight-control firmware. This approach allows for bidirectional autonomy within the constraints of the STM32's static routing, enabling the drone to receive sensor data and send navigation commands without any firmware modifications that could jeopardize flight stability. By embedding a simple type tag within the payload, the system effectively creates a lightweight protocol on top of CPX that distinguishes between different data types (e.g., scan data vs. waypoints) without requiring changes to the underlying routing logic. This design choice preserves the integrity of the flight control system while enabling flexible communication between the STM32 and GAP8, thereby ensuring that the autonomous navigation loop remains closed and responsive. Example code showing transmission and reception can be seen below,

```
uint8_t buf[1 + sizeof(WaypointPacket_t)];
buf[0] = PKT_TYPE_WAYPOINT;
memcpy(buf + 1, wp, sizeof(WaypointPacket_t));

cpxInitRoute(CPX_T_GAP8, CPX_T_STM32, CPX_F_APP, &txPacket.route);
memcpy(txPacket.data, buf, sizeof(buf));
txPacket.dataLength = sizeof(buf);
cpxSendPacketBlocking(&txPacket);
```

```
cpxInternalRouterReceiveOthers(&rxPacket);
// filter - only waypoint packets
if (rxPacket.dataLength < 1 + sizeof(WaypointPacket_t)) continue;
if (rxPacket.data[0] != PKT_TYPE_WAYPOINT) continue;
memcpy(&wp, rxPacket.data + 1, sizeof(WaypointPacket_t));
```

5.2 STM32 Firmware

The STM32 code developed for this pipeline exists within the app layer of the stock firmware so that it does not impact any of the low level controls like sensor fusion, state estimation, and message protocols. The code inside the app layer is responsible for reading ToF data from the Multi-ranger deck packaging it into the `ScanPacket_t` struct and sending it to the GAP8 to be used for mapping.

5.2.1 Processing ToF Data

The Multi-ranger deck reports a one-dimension scalar measurement between 5cm and 4m, this distance is then converted into a 2D point with respect to the drone's reference frame. This is done by multiplying the scalar value with the unit vector which corresponds with the given sensor direction. The following equations summarize the transformation, For any given sensor i, the local coordinate P_{local} is the product of the raw distance d_i and the orientation unit vector $\hat{\mathbf{u}}_i$:

$$\mathbf{P}_{local,i} = d_i \cdot \hat{\mathbf{u}}_i = d_i \cdot \begin{bmatrix} \hat{u}_{x,i} \\ \hat{u}_{y,i} \\ \hat{u}_{z,i} \end{bmatrix}$$

For the Rear (Back) sensor, your vector is defined as $\hat{\mathbf{u}}_{back} = [-1, 0, 0]^T$. If the sensor reports a distance $d = 1.5$m, the math expands to:

$$\mathbf{P}_{local,back} = 1.5 \cdot \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 0 \\ 0 \end{bmatrix}$$

This process is implemented in the firmware using a lookup table of unit vectors for each direction. These unit vectors are then used to transform each of the ToF readings from the five sensors into points in the drone frame. The look up tables can be seen in listing 1.

The lookup tables are then used to compute the x and y components for each ToF reading. Once computed they are packaged into the `ScanPacket_t`, along with the drone's current pose.

The populated `ScanPacket_t` is then sent using CPX to the GAP8 processor to be used for mapping and waypoint calculations

```
// x, y, z direction unit vectors for each sensor
// front: +x, back: -x, left: +y, right: -y, up: +z
static const float SENSOR_DX[5] = { 1.0f, -1.0f,  0.0f,  0.0f, 0.0f };
static const float SENSOR_DY[5] = { 0.0f,  0.0f,  1.0f, -1.0f, 0.0f };
static const float SENSOR_DZ[5] = { 0.0f,  0.0f,  0.0f,  0.0f, 1.0f };
```

Listing 1: Sensor lookup table implementation in `scan.c`.

5.2.2 Waypoint Processing

Once the GAP8 has computed a target frontier, as discussed in Section 5.4, it packages the destination into a `WaypointPacket_t` struct and sends it back to the STM32 over CPX. As discussed in Section 5.1, the waypoint packet is prefixed with a type tag `0x02` so the STM32 can distinguish it from incoming scan packets on the shared `mixedQueue`. The waypoint task blocks on `cpxInternalRouterReceiveOthers` and inspects the first byte of every arriving packet, discarding anything that is not a waypoint.

A design requirement was that autonomous navigation should not begin automatically on boot. To give the operator full control over when the drone begins following GAP8 waypoints, a Crazyflie parameter `mapper.autonomous` is exposed over the radio interface. This parameter defaults to `False` on every power cycle and must be explicitly set to `True` before the waypoint task will execute any navigation commands. It can be toggled at any time from the `cfclient` parameters tab or programmatically from the `fly.py` ground control script, allowing the operator to switch between manual and autonomous flight mid-session without landing or restarting the firmware.

When a valid waypoint is received and autonomous mode is enabled, the waypoint task first performs a directional obstacle check using the latest sensor readings. Unlike a simple minimum

range check against all sensors, the obstacle check projects the waypoint direction onto the drone's body frame axes and only flags a block if the sensor facing the direction of intended travel reads below the configured threshold. This prevents a wall behind or beside the drone from incorrectly blocking a move in the opposite direction, which was a common source of the drone getting stuck during early testing.

If the path is clear, the waypoint is executed in two phases. First the drone rotates to face the target heading using `crtpCommanderHighLevelGoTo` with a fixed pre-rotation duration, allowing the heading to stabilize before any translation begins. Testing showed that simultaneous yaw and position changes caused noticeable oscillation, so separating the two movements significantly improved flight smoothness. The drone then translates to the target position at a configured speed, waits for the move to complete, and optionally performs a spin on arrival in 90° increments. The spin allows the sensors to scan the surrounding area from a stationary position, densifying the map before the next frontier is computed. The key tunable parameters governing the behavior are summarized in Table 2.

Parameter	**Value**	**Description**
`FLIGHT_SPEED_S_PER_M`	8.0 s/m	Travel speed (0.125 m/s)
`PRE_ROTATE_DURATION`	2.5 s	Time to rotate before translating
`SPIN_STEPS`	2	Number of 90 increments on arrival
`SPIN_STEP_DURATION`	2.0 s	Duration of each spin step
`OBSTACLE_STOP_FRONT`	0.35 m	Front obstacle threshold
`OBSTACLE_STOP_SIDE`	0.20 m	Side obstacle threshold

Table 2: Key waypoint execution parameters defined in `app.c`.

5.3 GAP8 Firmware

The firmware running on the GAP8 is responsible for receiving `ScanPacket_t` from the STM32 then using them to develop the map. The map is then sent via the EPS32 to a client side script showing the current state of the map. Finally, a frontier based exploration is used to calculate the next waypoint the drone will fly to. This waypoint is packaged and sent back to the STM32 to fly the drone.

5.3.1 Occupancy Grid Mapping

The map is implemented using a 200×200 occupancy grid, with `int8_t` representing each cell. The cell resolution is 5cm each resulting in a 10mx10m mapping area and occupying 40kB. Instead of a binary value in each cell for a detection or not, a log-odds evidence system is used to ensure that any false detection or ToF noise does not directly result in a detection. As cells accumulate more ToF hits, they gain more confidence in being a detection. The map is stored in the L2 memory of the GAP8 due to its size and speed requirements; the L1 RAM is very fast but is only 64KB and mostly reserved for high speed computations.

The mapping procedure is initiated once it receives a `ScanPacket_t` from the STM32. The received ToF points are expressed as an (x, y) point in the drones local frame and must first be converted to the stationary map frame using a SE(2) rigid body transform according to the

following equations,

$$\begin{bmatrix} w_x \\ w_y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} p_x \\ p_y \end{bmatrix} + \begin{bmatrix} x_{drone} \\ y_{drone} \end{bmatrix} \tag{4}$$

Once computed these points in the global frame are discretized into a corresponding map cell using the `world_to_grid()` function. Since everything between the detected object and drone are free space, a ray can be casted between these points and all cells between are considered free. Bresenham's ray tracing algorithm is used to efficiently iterate through all the cells and update the confidence values. This procedure is discussed in more detail in section 5.3.2.

Once a map cell is changed, a dirty flag for that row is set to true. This is used to optimize the bandwidth of the map data being sent to the client. Only rows that have been modified will be sent, allowing for much faster updates to the client visualization.

A table summarizing the key parameters for the mapping algorithm can be seen in Table 3.

Parameter	**Value**	**Description**
`MAP_SIZE`	200 cells	Size of the occupancy grid
`MAP_RESOLUTION`	0.05 m/cell	Dimension of one grid cell
`MAP_ORIGIN`	100	Origin of the map in grid cells
`MAP_ORIGIN`	100	Origin of the map in grid cells
`CELL_MAX`	100	Maximum confidence value for a cell
`CELL_MIN`	-100	Minimum confidence value for a cell
`CELL_HIT`	10	Amount to increment cell value for a hit
`CELL_FREE`	-3	Amount to decrement cell value for a free space

Table 3: Key Mapping parameters defined in `map.c`

5.3.2 Bresenham's Ray Tracing

The line joining the the drone's pose and the ToF reading is not perfectly aligned within a discretized grid, because of this Bresenham's ray tracing algorithm is used to approximate this line into a discrete grid. It iterates through every grid cell along the straight-line path from the drone's current position (x_0, y_0) to the detected obstacle (x_1, y_1). For every intermediate cell, it applies a `CELL_FREE` penalty, increasing the confidence value along the path of any previous noise or moved objects. Only when it reaches the final coordinate (x_1, y_1) does it apply the `CELL_HIT` reward, incrementing the cell's value toward `CELL_MAX` to confirm the presence of a wall.

The function is highly optimized for the GAP8's embedded environment by using integer-only error tracking to avoid costly floating-point math during the loop. Furthermore, the inclusion of the `dirty[y0]` flag provides a critical performance optimization for the communication layer; by marking only the modified rows, the drone avoids re-transmitting static map data, significantly reducing the bandwidth required to stream the occupancy grid over the CPX protocol to your visualization tool. Pseudocode for the map updating algorithm is presented in algorithm 1.

Algorithm 1: Bresenham Ray Casting for Occupancy Grid Update

Input: Drone cell (x_0, y_0), Obstacle cell (x_1, y_1), Map M

Output: Updated Occupancy Grid M

```
dx ⟵ |x1 − x0|
dy ⟵ −|y1 − y0|
sx ⟵ (x0 < x1) ? +1 : −1
sy ⟵ (y0 < y1) ? +1 : −1
err ⟵ dx + dy

while True do
    if (x0, y0) out of grid bounds then
        break
    end
    if (x0, y0) = (x1, y1) then
        M[y0][x0] ⟵ min(M[y0][x0] + CELL_HIT, CELL_MAX)   // Mark occupied
        break
    else
        M[y0][x0] ⟵ max(M[y0][x0] + CELL_FREE, CELL_MIN)   // Mark free
         along ray
    end
    Dirty[y0] ⟵ True

    e2 ⟵ 2 × err
    if e2 ≥ dy then
        err ⟵ err + dy
        x0 ⟵ x0 + sx
    end
    if e2 ≤ dx then
        err ⟵ err + dx
        y0 ⟵ y0 + sy
    end
end
```

5.4 Frontier-Based Exploration

To fully map the environment the drone is in, an exploration algorithm that prioritizes exploring new areas is essential for optimal mapping. A frontier based algorithm is being used to send the drone to areas it has not seen yet. The exploration algorithm works by scanning every map cell and identifying any 'frontier cells'. A frontier cell is any unknown cell with an unexplored neighbor, examples can be seen in Figure 6. Once a list of frontier cells are generated, three sequential checks are done to see if the particular frontier cell is valid. First the candidate cell distance from the current pose is checked; there is a minimum distance threshold for the waypoint to encourage the drone to travel and see more of the environment. Next the candidate

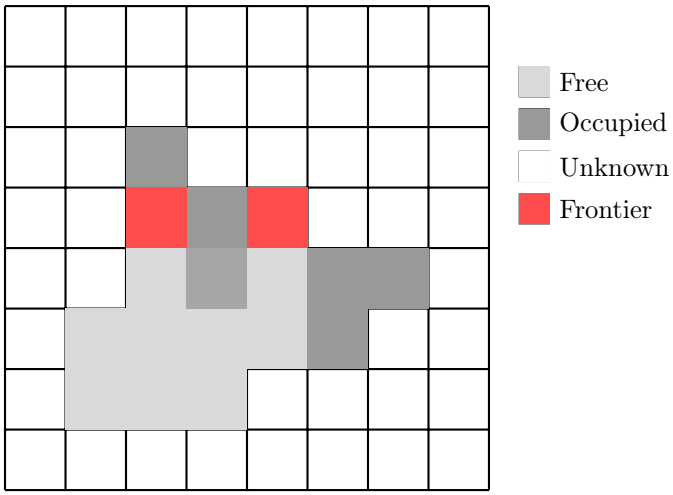

Figure 6: Diagram illustrating the concept of frontier cells in an occupancy grid. The red cells represent frontier cells, which are unknown cells adjacent to free space.

point is checked and ensured it is not too close to any detected objects. Finally, the straight line path between the candidate cell and the drone's current pose is calculated, if the path intercepts any occupied cell, the candidate is deemed invalid. Breshenham's line algorithm is used once again here to evaluate each of the cells along the path.

If a candidate cell passes all these conditions, the map cell is extracted and converted to an (x, y, yaw) pose in the map frame. Additionally, the `WaypointPacket_t` has field to indicate a waypoint is valid and a field to indicate if the drone should perform a spin upon arial at the waypoint. Currently, the `do_spin` field is always assigned to be true, but it was implemented for future flexibility.

Now that the `WaypointPacket_t` is populated with a target waypoint, it is sent back to the STM32 using CPX. The STM32 receives the waypoint and uses the commander module to first rotate the drone to the target yaw, then fly to the target waypoint pose. Through testing it was found that Crazyflie is unstable when translating and rotating at the same time. Once the drone reaches the target waypoint, it performs a 360 degree spin in the air to scan the environment around it, then the process is repeated with the drone sending a new `ScanPacket_t` to the GAP8 to update the map and calculate a new waypoint. This process continues until the drone has fully mapped its environment, meaning there are no more valid frontier cells to explore. A flowchart outlining the entire process can be seen in figure 7.

Although the frontier based exploration is effective at exploring simple environments, it is not perfect. The algorithm can get stuck in local minima, where the drone is surrounded by explored areas and cannot find a valid frontier cell to explore. Additionally, the algorithm does not take into account the drone's battery life or other constraints, which could lead to suboptimal exploration paths. Future work could involve implementing a more sophisticated exploration strategy that incorporates these factors, gradient potential fields, or RRT-based exploration.

A table summarizing the tunable parameters for the frontier-based exploration algorithm is shown in Table 4.

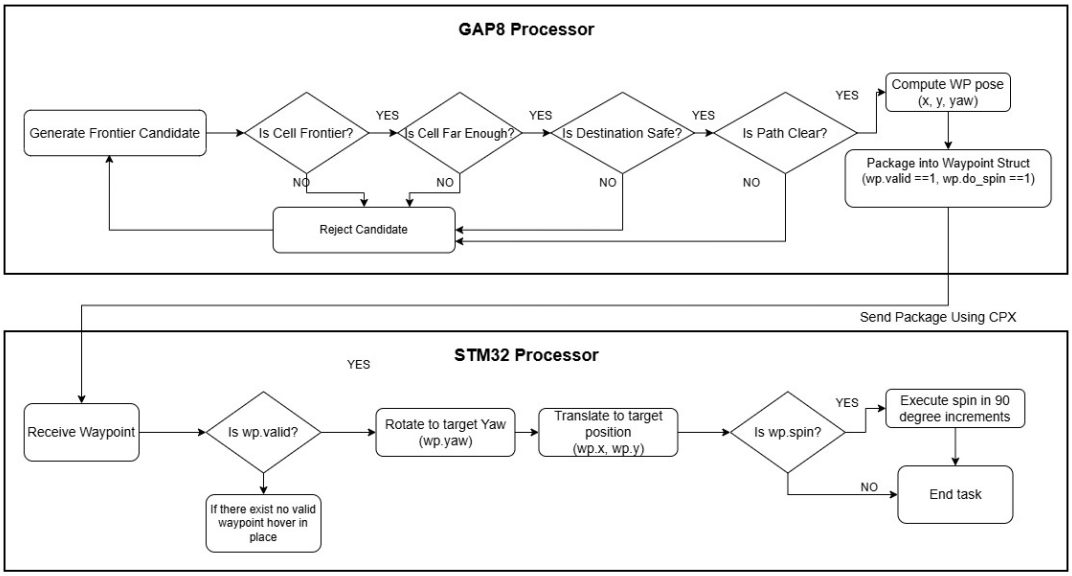

Figure 7: Flowchart outlining Frontier Based Exploration

Parameter	Value	Description
`MIN_FRONTIER_DIST_M`	0.8 m	Minimum distance for a candidate waypoint
`SAFE_RADIUS_CELLS`	6 cells	Number of cells around the waypoint to consider safe
`PATH_CLEARANCE_CELLS`	4 cells	Clearance needed around the drone's path

Table 4: Key Frontier-Based Exploration parameters defined in `frontier.c`

5.5 Host Software and ROS2 Integration

Since all mapping and frontier-based exploration are executed fully onboard the drone, the host software is responsible for real-time visualization; the host acts purely as an observer, receiving compact map and pose updates over WiFi and rendering them for the user.

To achieve this, a custom ROS2 node was developed in Python using the `rclpy` client library. The node interfaces directly with the ESP32 based communication system via UDP, allowing low-latency transmission of mapping data from the drone. Upon initialization, the node establishes a socket connection and sends a registration message (`FER`) to the ESP32 to begin streaming data. A dedicated background thread is used to continuously receive UDP packets, ensuring that network I/O does not block the ROS2 execution loop.

Incoming data packets follow CPX protocol, where each packet contains a function identifier distinguishing between map updates and pose updates. Map data is transmitted incrementally in row-wise chunks to reduce bandwidth usage. Each packet contains a row index and 200 signed 8-bit values representing the log-odds occupancy values for that row. A special sentinel value indicates when a full map update (delta) has been received, triggering publication to ROS2.

Internally, the node maintains a 2D grid storing these log-odds values. Before publishing, this grid is converted into a ROS2 compatible `nav_msgs/OccupancyGrid` topic. This involves both reshaping the data and remapping the log-odds representation into the ROS standard convention: unknown cells are assigned -1, free cells 0, and occupied cells 100. Additionally, proper map metadata such as resolution (0.05 m per cell), dimensions (200×200), and origin

Topic	Message type	Description
/cf/map	nav_msgs/OccupancyGrid	Full occupancy grid, updated on each map delta
/cf/pose	geometry_msgs/PoseStamped	Drone position and heading

Table 5: Crazyflie SLAM ROS2 topics

offset are configured so that the map aligns correctly in the global coordinate frame within RViz.

Pose data is transmitted separately as a set of floating-point values representing the drone's x, y, and yaw. Upon reception, this data is converted into a `geometry_msgs/PoseStamped` message and published for visualization. The yaw angle is converted into a quaternion representation to comply with ROS conventions. In addition to publishing the pose, the node also broadcasts a corresponding TF transform between the `map` frame and a `drone` frame. This allows RViz to render the drone with proper orientation and coordinate axes, providing a more intuitive understanding of its motion and heading. The RViz display can be seen in figure 8.

The separation of map and pose into two ROS2 topics enables flexible visualization and debugging. The occupancy grid is displayed as a 2D map, while the pose can be visualized as an arrow or coordinate frame overlay. This is particularly useful for diagnosing mapping issues such as drift, misalignment, or incorrect occupancy updates. The ROS2 topics used for visualization

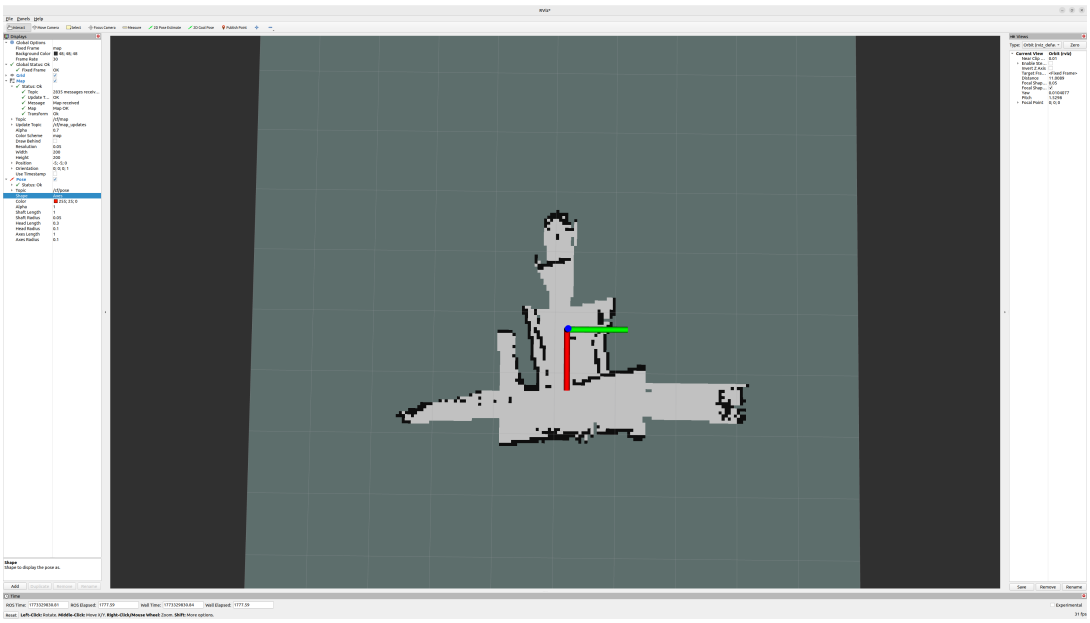

Figure 8: RViz visualization showing the occupancy grid map and drone pose. The map is rendered in grayscale, with occupied areas appearing darker and free space lighter. The drone's current pose is indicated by a axis marker aligned with its yaw.

are summarized in Table 5. This modular architecture ensures that the visualization pipeline remains simple, extensible, and decoupled from the onboard autonomy stack, while still providing rich real-time feedback during testing and development.

5.6 Repository Layout

The project source code is hosted at https://github.com/aowais2/crazyflie-slam and is organized into three modules, each corresponding to a distinct hardware layer in the system. The repository structure is shown in Figure 9 with the developed files marked with a '*'. The STM32 application lives in `stm32-sw/src/` and consists of three files: `scan.c` and `scan.h` handle ToF sensor processing and define the `ScanPacket_t` structure, while `app.c` contains the two FreeRTOS tasks responsible for packaging sensor data and executing navigation waypoints. The GAP8 application lives in `aideck-gap8-examples/examples/cf-mapping/` and contains all mapping and navigation logic; `map.c` and `map.h` implement the occupancy grid and Bresenham ray casting, `frontier.c` and `frontier.h` implement the frontier detection and waypoint generation, and `cf_mapping.c` is the main entry point coordinating the receive loop, map updates, and CPX communication. The surrounding `aideck-gap8-examples` repository is the standard Bitcraze example project and is included as the required build environment for the GAP SDK toolchain, with the custom application sitting inside it as a self-contained example. The host visualization lives in `ros2_ws/src/cf_mapping/` as a standard ROS2 Python package. The single file `mapper_node.py` handles all UDP communication with the drone and publishes the map and pose data for RViz2.

```
crazyflie-slam/
├── stm32-sw/
│   ├── src/
│   │   ├── app.c *
│   │   ├── scan.c *
│   │   └── scan.h *
│   ├── app-config
│   ├── Kbuild
│   └── Makefile
├── aideck-gap8-examples/
│   ├── examples/
│   │   └── cf-mapping/
│   │       ├── cf_mapping.c *
│   │       ├── map.c *
│   │       ├── map.h *
│   │       ├── frontier.c *
│   │       ├── frontier.h *
│   │       └── Makefile
│   └── lib/
│       └── cpx/
└── ros2_ws/
    └── src/
        └── cf_mapping/
            ├── mapper_node.py *
            ├── package.xml
            └── setup.py
```

Figure 9: Repository structure. Files marked with * are developed as part of this work.

5.7 Building and Deploying the Code

The deployment of the autonomous mapping system requires a synchronized build and flashing process across three distinct hardware layers: the GAP8 SoC on the AI Deck, the STM32 microcontroller on the Crazyflie 2.1, and the host-side ROS2 environment.

5.7.1 GAP8

The core occupancy grid logic and frontier finding algorithms are executed on the GAP8 RISC-V processor. Due to the complexity of the GAP-SDK dependencies, the firmware is compiled using a containerized Docker environment to ensure toolchain consistency.

The build process is initiated from the aideck-gap8-examples directory using the following command:

```
sudo docker run --rm -v ${PWD}:/module --privileged bitcraze/aideck \
  tools/build/make-example examples/cf-mapping clean all
```

Once the binary image is generated, it is flashed to the AI Deck over-the-air (OTA) via the Crazyradio PA. The `cfloader` utility targets the `aideck-bcAI` memory bank:

```
cfloader flash \
  examples/cf-mapping/BUILD/GAP8_V2/GCC_RISCV_FREERTOS/target.board.
     devices.flash.img \
  deck-bcAI:gap8-fw -w radio://0/80/2M/[DRONE_URI]
```

5.7.2 STM32

The STM32 manages the low-level flight control and coordinates communication between the expansion decks and the primary MCU. This firmware is compiled natively on the Ubuntu host using GNU Make. To buid the STM32 firmware, navigate to the crazyflie-firmware directory and run:

```
make -j$(nproc)
```

After ensuring the drone is in bootloader mode, the firmware is uploaded using the Bitcraze `cload` utility:

```
cload flash
```

5.7.3 ESP32

As mentioned in 4.4 the ESP32 firmware is a critical component of the communication bridge and must be flashed with the optimized LARICS firmware to ensure stability. The LARICS ESP32 firmware can be built by running the following command in the root of the forked repository:

```
        docker run --rm -it -v $PWD:/module/ --device /dev/ttyUSB0 --
           privileged -P bitcraze/builder /bin/bash -c "source /
           new_home/.espressif/python_env/idf4.3_py3.10_env/bin/
           activate && make"
```

And it can be flashed using the following command:

```
        cfloader flash build/aideck_esp.bin deck-bcAI:esp-fw -w radio
           ://0/90/2M/[DRONE_URI]
```

Further details on the ESP32 firmware and the LARICS optimization [9] can be found in Section 4.4.

5.7.4 Client Side ROS2 Vizualizer

The mapping data is visualized and logged on a host computer running ROS2 Humble. The `ros2_ws` workspace contains the `mapper_node`, which processes incoming UDP packets and converts them into standard ROS2 messages. To build the ROS2 node, navigate to the `ros2_ws` directory and run:

```
colcon build
```

To run the ROS2 node, source the workspace and launch the mapper node:

```
source install/setup.bash
ros2 run mapper_node mapper_node
```

The RViz visualization can be launched separately to monitor the occupancy grid and drone pose in real-time,

```
ros2 run RViz2 RViz2
```

5.7.5 Running the Mapping and Exploration

Once the firmware is flashed to the drone and the ROS2 node is running, the drone can be powered on to begin the mapping and exploration process. The drone's initial start up position will be considered the origin of the map. From `cfclient` the drone can be armed and takeoff can be initiated. Once the drone is in the air, the autonomous navigation can be enabled by setting the `mapper.autonomous` parameter to true. The drone will then begin sending ToF data to the GAP8, which will build the map and calculate waypoints for the drone to fly to. The drone will continue this process until it has fully mapped its environment, meaning there are no more valid frontier cells to explore.

5.8 Evaluation and Results

The mapping and exploration pipeline was tested in a controlled indoor environment with a few small obstacles. The drone was able to successfully build a map of the environment and navigate to new areas using the frontier based exploration algorithm. The occupancy grid map was visualized in real time using RViz, allowing for monitoring of the mapping progress. Due to the very simple path planning used, the drone would often get stuck in local minima, where it would be surrounded by explored areas and could not find a valid frontier cell to explore. Additionally, due to not having any immediate service interrupts for the STM32 it was subject to hit objects. Drift in the odometry over time would cause the drone to misjudge its position and

hit objects, which would cause the drone to crash. The drone also had a tendency to get stuck in corners or against walls, where it would continuously try to fly into the wall and eventually crash. These frequent crashes were a significant challenge during testing and development, as they would interrupt the workflow and require time-consuming recovery procedures. Despite these challenges, the drone was able to ocasionally successfully map the environment and navigate to new areas, demonstrating the viability of the onboard mapping and exploration approach. However, the limitations of the hardware and the simplicity of the exploration algorithm meant that the performance was not consistent, and there were many cases where the drone would fail to explore effectively or would crash due to odometry drift, crashing, getting stuck, or other issues.

6 Hardware Limitations and Challenges

The Crazyflie 2.1 is an acceptable research platform for its intended purpose, rapid prototyping and algorithm validation on a small, accessible drone. However, pushing it beyond its intended use case into a fully autonomous mapping and exploration system exposed a number of significant hardware and software limitations that had a direct and recurring impact on the development process.

6.1 Flight Dynamics

The most immediately apparent physical limitation is weight distribution. The Crazyflie 2.1 is designed and tuned as a bare platform, and adding three expansion decks simultaneously shifts the centre of mass significantly while adding approximately 16g to a platform that weighs only 29g stock. The Flow deck below, Multi-ranger deck above, and AI deck stacked on top are not evenly distributed, and the stock PID controller is tuned for the bare airframe. The result is noticeably degraded attitude stability, particularly during yaw maneuvers and when recovering from disturbances. The drone is very prone to crashing when trying to perform very simple turns or translations, and the risk of crashing is significantly higher when the battery voltage begins to drop. This instability not only limits the drone's ability to navigate effectively but also leads to frequent crashes that interrupt testing and development.

6.2 Battery Life

Closely related is the power delivery system. The Crazyflie runs on a single-cell 100mAh LiPo battery which is already marginal for the stock platform. With three additional decks drawing current, particularly the AI deck with its GAP8 and ESP32 both active, battery life drops to approximately $5 - 7$ minutes under normal flight conditions. This is barely sufficient for a meaningful mapping run, and in practice much of that time is consumed by the spin maneuvers and slow waypoint travel that are necessary for good map coverage. The electronic speed controllers are similarly not rated for the additional payload, and under aggressive corrections the motors do not respond with the authority needed to maintain stable attitude, particularly when the battery voltage begins to sag toward the end of a flight.

6.3 Sensor Sparsity

The sensor capability is a fundamental constraint that shapes the entire mapping architecture. Each VL53L1x sensor on the Multi-ranger deck returns a single scalar distance, one number per direction per scan. NanoSLAM, which this work is directly inspired by, used the VL53L5CX which provides an 8×8 zone matrix per sensor yielding 32 usable points per scan. The difference in map density is substantial. With only four horizontal points per scan the occupancy grid builds up slowly, walls appear as sparse dotted lines rather than solid boundaries, and the frontier detection algorithm tends to find many small targets close to the drone rather than identifying genuinely unexplored regions further away. Flying slowly and performing spin maneuvers on arrival partially compensates for this, but it cannot fully overcome the fundamental sparsity of single-point sensors. Additionally, the lack of reported features or intensity values from the sensors means that scan matching and loop closure techniques are not currently feasible, which contributes to significant pose drift over time.

6.4 Compute Headroom

Compute capability on the GAP8 is adequate for the current pipeline but leaves limited headroom for future development. The 512KB of L2 RAM must accommodate the 40KB occupancy grid, CPX communication buffers, the FreeRTOS kernel, and the full GAP SDK runtime simultaneously. Adding ICP scan matching, which is the natural next step to address pose drift, would require storing multiple scan histories and performing iterative matrix operations within the remaining memory budget. This is theoretically feasible but requires very careful memory management and pushes the platform close to its limits.

6.5 Practical Applications

The inability to fly outdoors is a practical constraint that limits where testing can be conducted. The expansion decks are open PCBs with no enclosure or weather protection, meaning any air movement, humidity, or debris immediately threatens the electronics. The optical flow sensor additionally requires a flat, textured surface at a consistent altitude to function correctly, making outdoor use impractical even in calm conditions.

6.6 Software Ecosystem

One of the most frustrating recurring challenges during development was the AI deck flashing process. Flashing new GAP8 firmware requires the drone to be powered on with the Crazyradio connected, and the process is sensitive to timing and connection stability. A failed flash, which occurs frequently enough to be a routine occurrence rather than an exception, corrupts the AI deck bootloader state and leaves the deck completely unresponsive. Recovery requires flashing the stock Bitcraze firmware back to the STM32 first, then reflashing the AI deck with stock firmware to restore a known good state, and then rebuilding and reflashing the custom GAP8 application from scratch. Each complete recovery cycle takes 15 to 20 minutes and interrupts the development workflow significantly. Over the course of the project this added up to a considerable amount of lost time.

Additionally, GreenWaves Technologies, the company behind the GAP8 processor and its SDK, is no longer actively developing the platform. The GAP SDK that this entire project depends on is effectively abandoned with no supported upgrade path available. The bootloader toolchain has not received official updates and relies on patches contributed by individual community members rather than a maintained release. This creates a fragile dependency where if a breaking change appears in the Linux toolchain or Docker environment there is no guarantee that a fix will be forthcoming from the vendor.

7 Conclusion

This thesis set out to demonstrate that a complete mapping and autonomous exploration pipeline could run entirely onboard a nano-UAV with no external infrastructure, no offboard compute, and no prior knowledge of the environment. That goal was achieved. The implemented system successfully builds a 2D log-odds occupancy grid in real time on the GAP8 processor, streams it to a ground station over WiFi, and autonomously navigates toward unexplored frontiers using only the onboard sensor suite and EKF pose estimate. The drone is capable of taking off, mapping its surroundings, identifying frontier cells, and flying to them without any operator input beyond the initial arming and autonomous mode enable. This represents a meaningful step toward the fully decentralized autonomous swarm operation described in the C-SLAM framework that motivated this work.

However, the honest assessment is that the Crazyflie 2.1 platform, while invaluable for rapid prototyping and algorithm development, is not the right vehicle for pushing this work further. The fundamental limitations encountered throughout this project are not software problems that can be fixed with better code. They are hardware constraints that the platform was simply not designed to accommodate. A detailed analysis of these limitations and their impact on the project is discused in Section 6, but the key issues can be summarized as follows:

- Flight dynamics are severely degraded under the weight of the expansion decks, leading to frequent crashes and unstable flight that impacts mapping quality.
- The sensor suite is too sparse to produce dense maps or enable scan matching, resulting in significant pose drift and smeared maps over time.
- Compute headroom on the GAP8 is limited, making it challenging to implement more sophisticated algorithms like ICP without careful memory management.
- The software ecosystem, particularly for the GAP8, is fragile and unmaintained
- The platform is not suitable for outdoor use, limiting testing environments and real-world applicability.
- The flashing process for the AI deck is fragile and prone to failure, leading to frequent interruptions in the development workflow.

Due to these fundamental constraints the recommended path forward for extending this work toward the C-SLAM swarm coordination objectives described in the research objectives

is to migrate to a custom drone platform. 3D printed frames allow the sensor and compute payload to be positioned for optimal weight distribution rather than stacked in whatever order the Crazyflie deck connector permits. Off-the-shelf brushless motors and ESCs sized for the actual flying weight would restore the attitude authority that the Crazyflie loses under payload. A more capable single-board computer such as a Raspberry Pi Zero 2W or equivalent running ROS2 natively would replace the GAP8 and ESP32 with a single Linux system that has a maintained software ecosystem, native WiFi, and enough memory to implement full ICP and pose graph optimization without the tight constraints of 512KB L2 RAM. The software architecture developed in this thesis transfers directly: the occupancy grid representation, Bresenham ray casting, log-odds update model, frontier detection algorithm, and delta encoding communication scheme are all hardware agnostic and would run without modification on any Linux system. The inter-processor communication layer would simplify considerably since a single Linux board eliminates the STM32 to GAP8 CPX bridge entirely. Custom platforms of this type are also substantially cheaper per unit than purchasing Bitcraze hardware, making the larger swarm deployments required for C-SLAM coordination experiments economically practical.

In summary, this work validates the core algorithmic approach: onboard log-odds occupancy grid mapping with Bresenham ray casting and frontier-based exploration is feasible on constrained nano-UAV hardware and produces navigable maps of indoor environments without external infrastructure. The Crazyflie platform served its purpose as a development and validation vehicle. The next phase of this research should be carried out on custom hardware that removes the mechanical, electrical, and software constraints that limited what could be demonstrated here, and with VL53L5CX matrix sensors that provide the geometric richness required to close the SLAM loop with ICP.

References

[1] Bitcraze AB. Multi-ranger deck. https://www.bitcraze.io/products/multi-ranger-deck/, 2018. [Accessed: Oct. 11, 2025].

[2] Bitcraze AB. *AI Deck 1.1: Nano-UAV Intelligence Board*. Bitcraze AB, Malmö, Sweden, 2026. Accessed: 2026-04-03.

[3] Bitcraze AB. *Crazyflie 2.1 Open-Source Nano Quadcopter*. Bitcraze AB, Malmö, Sweden, 2026. Accessed: 2026-04-03.

[4] Bitcraze AB. Crazyflie python client library. https://github.com/bitcraze/crazyflie-lib-python, 2026. Accessed: 2026-04-03.

[5] C. Friess, V. Niculescu, T. Polonelli, M. Magno, and L. Benini. Fully onboard slam for distributed mapping with a swarm of nano-drones. *IEEE Internet of Things Journal*, 2025. To appear.

[6] V. Gu. Implementation of probabilistic search algorithm using crazyflie swarm uav, 2025.

[7] Vlad Niculescu, Tommaso Polonelli, Michele Magno, and Luca Benini. Nanoslam: Enabling fully onboard slam for tiny robots. *IEEE Internet of Things Journal*, 2023.

[8] Vlad Niculescu, Tommaso Polonelli, Michele Magno, and Luca Benini. Ultra-lightweight collaborative slam for robot swarms. *IEEE Access*, 2025.

[9] Marijana Peti and Laboratory for Robotics and Intelligent Control Systems (LARICS). Optimized ESP32 WiFi bridge and GAP8 streaming for AI-Deck. https://github.com/marijana23/esp-idf-bridge, 2022. University of Zagreb. Special firmware optimization for high-throughput SPI-to-UDP bridging.

Appendices

A STM32 Developed Code

Listing 2: scan.c

```c
#include "scan.h"
#include <math.h>
#include <string.h>
#define DEBUG_MODULE "SCAN"
#include "debug.h"

#define MAX_RANGE_MM 3000
#define MIN_RANGE_MM 50


// <*LUT_ARRAYS*>
// x, y, z direction unit vectors for each sensor
// front: +x, back: -x, left: +y, right: -y, up: +z
static const float SENSOR_DX[5] = { 1.0f, -1.0f,  0.0f,  0.0f, 0.0f };
static const float SENSOR_DY[5] = { 0.0f,  0.0f,  1.0f, -1.0f, 0.0f };
static const float SENSOR_DZ[5] = { 0.0f,  0.0f,  0.0f,  0.0f, 1.0f };
// <*LUT_ARRAYS*>
static const char *SENSOR_NAMES[5] = {
    "FRONT", "BACK", "LEFT", "RIGHT", "UP"
};

void scan_extract_points(uint16_t ranges[5], ScanPacket_t *scan)
{
    static uint32_t call_count = 0;
    call_count++;
    uint8_t do_print = 0; //(call_count % 10 == 0);

    // Zero entire struct so unused slots are not A5 garbage
    memset(scan, 0, sizeof(ScanPacket_t));

    if (do_print) {
        DEBUG_PRINT("--- scan ---\n");
        DEBUG_PRINT("raw: F=%4dmm B=%4dmm L=%4dmm R=%4dmm U=%4dmm\n",
                    ranges[0], ranges[1], ranges[2], ranges[3], ranges
                        [4]);
    }
// <*PROJECTION_LOOP*>
    for (int i = 0; i < 5; i++) {
        uint16_t r = ranges[i];
        float dist = (float)r / 1000.0f;
        scan->x[scan->num_points] = dist * SENSOR_DX[i];
        scan->y[scan->num_points] = dist * SENSOR_DY[i];
        scan->z[scan->num_points] = dist * SENSOR_DZ[i];
// <*PROJECTION_LOOP*>
```

```
        if (do_print)
            DEBUG_PRINT("  [%5s] %5dmm -> (%.3f, %.3f, %.3f) m\n",
                        SENSOR_NAMES[i], r,
                        (double)scan->x[scan->num_points],
                        (double)scan->y[scan->num_points],
                        (double)scan->z[scan->num_points]);

        scan->num_points++;
    }

    if (do_print)
        DEBUG_PRINT("points: %d valid\n", scan->num_points);
}
```

Listing 3: scan.h

```c
#pragma once
#include <stdint.h>

#define MAX_POINTS 5

// <*ScanPacket*>
typedef struct __attribute__((packed)) {
    uint8_t num_points;
    float   x[MAX_POINTS];
    float   y[MAX_POINTS];
    float   z[MAX_POINTS];
    float   pose_x;
    float   pose_y;
    float   pose_yaw;
} ScanPacket_t;
// <*ScanPacket*>
// Verify size at compile time - should be 1 + 20 + 20 + 20 + 12 = 73 bytes
_Static_assert(sizeof(ScanPacket_t) == 73, "ScanPacket_t size mismatch");

void scan_extract_points(uint16_t ranges[5], ScanPacket_t *scan);
```

Listing 4: app.c

```c
#include "app.h"
#include "cpx.h"
#include "cpx_internal_router.h"
#include "FreeRTOS.h"
#include "task.h"
#include "log.h"
#include "crtp_commander_high_level.h"
#include "param.h"
#include <math.h>
#include <string.h>

#include "scan.h"

#define DEBUG_MODULE "MAPPER"
#include "debug.h"

//  constants
#define CPX_F_APP            5
#define PKT_TYPE_WAYPOINT    0x02
#define FLIGHT_SPEED_S_PER_M 8.0f    // 0.125 m/s
#define TURN_SPEED_S_PER_M 10.0f
#define MAX_FLIGHT_DURATION  20.0f   // seconds cap
#define PRE_ROTATE_DURATION  2.5f    // rotate before translating
#define HOVER_Z              0.5f
#define SPIN_STEPS           2       // 2 x 90 = 180 degrees
#define SPIN_STEP_DEG        90.0f
#define SPIN_STEP_DURATION   2.0f    // seconds per 90 degree turn
#define SPIN_SETTLE_MS       500    // ms to wait after each spin step

// per-direction obstacle thresholds
#define OBSTACLE_STOP_FRONT  0.35f
#define OBSTACLE_STOP_BACK   0.20f
#define OBSTACLE_STOP_SIDE   0.20f

//  autonomous param
static uint8_t autonomous_enabled = 0;

PARAM_GROUP_START(mapper)
PARAM_ADD(PARAM_UINT8, autonomous, &autonomous_enabled)
PARAM_GROUP_STOP(mapper)

//  shared state
static volatile float current_range_front = 9999.0f;
static volatile float current_range_back  = 9999.0f;
static volatile float current_range_left  = 9999.0f;
static volatile float current_range_right = 9999.0f;
static volatile float current_pose_x      = 0.0f;
```

```
static volatile float current_pose_y      = 0.0f;
static volatile float current_pose_yaw    = 0.0f;

//  waypoint struct  must match frontier.h on GAP8
typedef struct __attribute__((packed)) {
    float   x;
    float   y;
    float   z;
    float   yaw;
    uint8_t valid;
    uint8_t do_spin;
} WaypointPacket_t;

//  task 1: scan sender

static void scanTask(void *param)
{
    (void)param;
    vTaskDelay(M2T(2000));
    DEBUG_PRINT("Scan task ready\n");

    logVarId_t idFront = logGetVarId("range", "front");
    logVarId_t idBack  = logGetVarId("range", "back");
    logVarId_t idLeft  = logGetVarId("range", "left");
    logVarId_t idRight = logGetVarId("range", "right");
    logVarId_t idUp    = logGetVarId("range", "up");
    logVarId_t idX     = logGetVarId("stateEstimate", "x");
    logVarId_t idY     = logGetVarId("stateEstimate", "y");
    logVarId_t idYaw   = logGetVarId("stateEstimate", "yaw");

    CPXPacket_t txPacket;
    cpxInitRoute(CPX_T_STM32, CPX_T_GAP8,
                 CPX_F_APP, &txPacket.route);

    ScanPacket_t scan;
    uint32_t packet_count = 0;
    TickType_t lastWake = xTaskGetTickCount();

    while (1) {
        uint16_t ranges[5] = {
            (uint16_t)logGetUint(idFront),
            (uint16_t)logGetUint(idBack),
            (uint16_t)logGetUint(idLeft),
            (uint16_t)logGetUint(idRight),
            (uint16_t)logGetUint(idUp),
        };

        current_range_front = ranges[0] / 1000.0f;
```

```
        current_range_back  = ranges[1] / 1000.0f;
        current_range_left  = ranges[2] / 1000.0f;
        current_range_right = ranges[3] / 1000.0f;

        scan_extract_points(ranges, &scan);

        scan.pose_x   = logGetFloat(idX);
        scan.pose_y   = logGetFloat(idY);
        scan.pose_yaw = logGetFloat(idYaw) * (float)M_PI / 180.0f;

        current_pose_x   = scan.pose_x;
        current_pose_y   = scan.pose_y;
        current_pose_yaw = scan.pose_yaw;

        if (packet_count % 10 == 0) {
            DEBUG_PRINT("pose:(%.2f,%.2f,%.1fdeg) pts=%d pkt#%lu\n",
                        (double)scan.pose_x,
                        (double)scan.pose_y,
                        (double)(scan.pose_yaw * 180.0f / (float)M_PI),
                        scan.num_points,
                        packet_count);
        }

        memcpy(txPacket.data, &scan, sizeof(ScanPacket_t));
        txPacket.dataLength = sizeof(ScanPacket_t);
        cpxSendPacketBlocking(&txPacket);

        packet_count++;
        vTaskDelayUntil(&lastWake, M2T(100));
    }
}

//  spin in place

static void do_spin(float arrival_yaw, float x, float y)
{
    DEBUG_PRINT("Spinning %d steps\n", SPIN_STEPS);

    for (int step = 1; step <= SPIN_STEPS; step++) {
        float target_yaw = arrival_yaw +
                           (SPIN_STEP_DEG * (float)M_PI / 180.0f) *
                              step;

        // relative=true, zero x,y,z - only yaw changes
        // Flow deck holds position naturally, no EKF fighting
        crtpCommanderHighLevelGoTo(
            x, y, HOVER_Z,
            target_yaw,
```

```
            SPIN_STEP_DURATION,
            false
        );

        vTaskDelay(M2T((uint32_t)(SPIN_STEP_DURATION * 1000.0f)
                       + SPIN_SETTLE_MS));

        DEBUG_PRINT("Spin step %d/%d done\n", step, SPIN_STEPS);
    }

    DEBUG_PRINT("Spin complete\n");
}

// task 2: waypoint receiver

static void waypointTask(void *param)
{
    (void)param;
    vTaskDelay(M2T(3000));
    DEBUG_PRINT("Waypoint task ready\n");

    CPXPacket_t rxPacket;
    WaypointPacket_t wp;

    while (1) {

                // <*STM32_REC*>
                cpxInternalRouterReceiveOthers(&rxPacket);
                // filter - only waypoint packets
                if (rxPacket.dataLength < 1 + sizeof(WaypointPacket_t))
                    continue;
                if (rxPacket.data[0] != PKT_TYPE_WAYPOINT) continue;
                memcpy(&wp, rxPacket.data + 1, sizeof(WaypointPacket_t)
                    );
                // <*STM32_REC*>
        if (!wp.valid) {
            DEBUG_PRINT("No frontier - hovering\n");
            continue;
        }

        if (!autonomous_enabled) {
            DEBUG_PRINT("Autonomous disabled\n");
            continue;
        }

        // directional obstacle check - only block if obstacle is
        // in the direction we are actually flying toward
        float dx      = wp.x - current_pose_x;
```

```
        float dy      = wp.y - current_pose_y;
        float yaw     = current_pose_yaw;
        float forward = dx * cosf(yaw) + dy * sinf(yaw);
        float lateral = -dx * sinf(yaw) + dy * cosf(yaw);

        int blocked = 0;

        if (forward > 0.1f && current_range_front < OBSTACLE_STOP_FRONT) {
            DEBUG_PRINT("Blocked␣FRONT␣%.3fm\n",
                        (double)current_range_front);
            blocked = 1;
        }
        if (forward < -0.1f && current_range_back < OBSTACLE_STOP_BACK) {
            DEBUG_PRINT("Blocked␣BACK␣%.3fm\n",
                        (double)current_range_back);
            blocked = 1;
        }
        if (lateral > 0.1f && current_range_left < OBSTACLE_STOP_SIDE) {
            DEBUG_PRINT("Blocked␣LEFT␣%.3fm\n",
                        (double)current_range_left);
            blocked = 1;
        }
        if (lateral < -0.1f && current_range_right < OBSTACLE_STOP_SIDE) {
            DEBUG_PRINT("Blocked␣RIGHT␣%.3fm\n",
                        (double)current_range_right);
            blocked = 1;
        }

        if (blocked) {
            DEBUG_PRINT("Obstacle␣in␣flight␣direction␣-␣skipping\n");
            continue;
        }

        // calculate flight duration
        float dist     = sqrtf(dx*dx + dy*dy);
        float duration = dist * FLIGHT_SPEED_S_PER_M;
        if (duration < 2.0f)                duration = 2.0f;
        if (duration > MAX_FLIGHT_DURATION) duration = MAX_FLIGHT_DURATION;

        DEBUG_PRINT("->␣(%.2f,%.2f)␣yaw=%.1fdeg␣dist=%.2fm␣dur=%.1fs\n",
                    (double)wp.x, (double)wp.y,
                    (double)(wp.yaw * 180.0f / (float)M_PI),
```

```
                    (double)dist, (double)duration);

        // step 1 - rotate to face frontier before moving
        crtpCommanderHighLevelGoTo(
            current_pose_x, current_pose_y,
            HOVER_Z, wp.yaw,
            PRE_ROTATE_DURATION, false
        );
        vTaskDelay(M2T((uint32_t)(PRE_ROTATE_DURATION * 1000.0f) + 500)
            );

        // step 2 - fly to frontier
        crtpCommanderHighLevelGoTo(
            wp.x, wp.y, HOVER_Z,
            wp.yaw, duration, false
        );
        vTaskDelay(M2T((uint32_t)(duration * 1000.0f)));

        // step 3 - spin on arrival if requested
        if (wp.do_spin) {
            do_spin(wp.yaw, wp.x, wp.y);
        }

        // brief settle before next waypoint
        vTaskDelay(M2T(1000));
    }
}

// entry point

void appMain()
{
    DEBUG_PRINT("Starting mapper + autonomous nav\n");
    xTaskCreate(scanTask,     "scanTask",     300, NULL, 2, NULL);
    xTaskCreate(waypointTask, "waypointTask", 300, NULL, 2, NULL);
    DEBUG_PRINT("Created tasks\n");
}
```

B AI Deck Developed Code

Listing 5: cf_mapping.c

```c
#include "pmsis.h"
#include "bsp/bsp.h"
#include "cpx.h"
#include "map.h"
#include "frontier.h"
#include <string.h>
#include <math.h>

#define MAX_POINTS 5

// structs

typedef struct __attribute__((packed)) {
    uint8_t num_points;
    float   x[MAX_POINTS];
    float   y[MAX_POINTS];
    float   z[MAX_POINTS];
    float   pose_x;
    float   pose_y;
    float   pose_yaw;
} ScanPacket_t;

typedef struct __attribute__((packed)) {
    uint16_t chunk_index;
    uint8_t  row;
    int8_t   data[MAP_SIZE];
} MapChunk_t;

typedef struct __attribute__((packed)) {
    float pose_x;
    float pose_y;
    float pose_yaw;
    float dist_front;
    float dist_back;
    float dist_left;
    float dist_right;
} PosePacket_t;

// CPX IDs
#define CPX_F_APP    5
#define CPX_F_MAP   10
#define CPX_F_POSE  11

// constants
```

```
#define MAP_DELTA_END      255
#define FRONTIER_INTERVAL  100     // every 30 scans = 3 seconds
#define HOVER_Z            0.25f   // must match STM32 takeoff height
#define PKT_TYPE_WAYPOINT  0x02   // type tag for outgoing waypoint
    packets

// state
static CPXPacket_t rxPacket;
static CPXPacket_t txPacket;
PI_L2 OccupancyMap_t map;
uint8_t dirty[MAP_SIZE];

// send_map_delta

static void send_map_delta(void)
{
    MapChunk_t chunk;
    uint16_t chunk_idx = 0;

    for (int row = 0; row < MAP_SIZE; row++) {
        if (!dirty[row]) continue;
        dirty[row] = 0;

        chunk.chunk_index = chunk_idx++;
        chunk.row         = (uint8_t)row;
        memcpy(chunk.data, map.cells[row], MAP_SIZE);
                // <*WIFI_SEND*>
        cpxInitRoute(CPX_T_GAP8, CPX_T_WIFI_HOST,
                     CPX_F_MAP, &txPacket.route);
        memcpy(txPacket.data, &chunk, sizeof(MapChunk_t));
        txPacket.dataLength = sizeof(MapChunk_t);
        cpxSendPacketBlocking(&txPacket);
                // <*WIFI_SEND*>
    }

    // sentinel - tells Python to redraw
    chunk.chunk_index = chunk_idx;
    chunk.row         = MAP_DELTA_END;
    memset(chunk.data, 0, MAP_SIZE);
    cpxInitRoute(CPX_T_GAP8, CPX_T_WIFI_HOST,
                 CPX_F_MAP, &txPacket.route);
    memcpy(txPacket.data, &chunk, sizeof(MapChunk_t));
    txPacket.dataLength = sizeof(MapChunk_t);
    cpxSendPacketBlocking(&txPacket);
}

// send_pose

```

```c
static void send_pose(float px, float py, float pyaw, ScanPacket_t *
    scan)
{
    PosePacket_t p = {0};
    p.pose_x   = px;
    p.pose_y   = py;
    p.pose_yaw = pyaw;

    for (int i = 0; i < scan->num_points; i++) {
        if (scan->z[i] > 0.01f) continue;
        float d = sqrtf(scan->x[i]*scan->x[i] + scan->y[i]*scan->y[i]);
        if      (scan->x[i] >  0.01f) p.dist_front = d;
        else if (scan->x[i] < -0.01f) p.dist_back  = d;
        else if (scan->y[i] >  0.01f) p.dist_left  = d;
        else if (scan->y[i] < -0.01f) p.dist_right = d;
    }

    cpxInitRoute(CPX_T_GAP8, CPX_T_WIFI_HOST,
                 CPX_F_POSE, &txPacket.route);
    memcpy(txPacket.data, &p, sizeof(PosePacket_t));
    txPacket.dataLength = sizeof(PosePacket_t);
    cpxSendPacketBlocking(&txPacket);
}

//send_waypoint

static void send_waypoint(WaypointPacket_t *wp)
{
    // prefix with type tag  STM32 uses this to identify waypoint
        packets
    // since STM32 cannot filter by CPX function ID

        // <*STM32_SEND*>
    uint8_t buf[1 + sizeof(WaypointPacket_t)];
    buf[0] = PKT_TYPE_WAYPOINT;
    memcpy(buf + 1, wp, sizeof(WaypointPacket_t));

    cpxInitRoute(CPX_T_GAP8, CPX_T_STM32,
                 CPX_F_APP, &txPacket.route);
    memcpy(txPacket.data, buf, sizeof(buf));
    txPacket.dataLength = sizeof(buf);
    cpxSendPacketBlocking(&txPacket);
        // <*STM32_SEND*>
}

//main loop

void start_example(void)
```

```
{
    pi_bsp_init();
    cpxInit();
    cpxEnableFunction(CPX_F_APP);

    pi_time_wait_us(500 * 1000);
    cpxPrintToConsole(LOG_TO_CRTP, "CF-Mapper␣GAP8␣ready");
    cpxPrintToConsole(LOG_TO_CRTP, "heap␣before␣ibit");
    map_init(&map);
    cpxPrintToConsole(LOG_TO_CRTP, "heap␣after␣ibit");
    memset(dirty, 0, sizeof(dirty));

    uint32_t scan_count = 0;
    ScanPacket_t scan;

    while (1) {
        // receive scan from STM32 - unchanged, working
        cpxReceivePacketBlocking(CPX_F_APP, &rxPacket);
        memcpy(&scan, rxPacket.data, sizeof(ScanPacket_t));

        // update map
        map_update(&map,
                   scan.pose_x, scan.pose_y, scan.pose_yaw,
                   scan.x, scan.y, scan.z,
                   scan.num_points);

        // debug log
        // if (scan_count % 10 == 0) {
        //     cpxPrintToConsole(LOG_TO_CRTP,
        //         "scan #%u pts=%u pose=(%.2f,%.2f)",
        //         (unsigned)scan_count,
        //         (unsigned)scan.num_points,
        //         (double)scan.pose_x,
        //         (double)scan.pose_y);
        // }

        // send pose + map delta every scan
        send_pose(scan.pose_x, scan.pose_y, scan.pose_yaw, &scan);
        send_map_delta();

        // frontier detection every FRONTIER_INTERVAL scans
        if (scan_count % FRONTIER_INTERVAL == 0) {
            WaypointPacket_t wp;
            int found = frontier_find(&map,
                                      scan.pose_x, scan.pose_y,
                                      &wp, HOVER_Z);
            if (found) {
                cpxPrintToConsole(LOG_TO_CRTP,
```

```
                    "frontier: (%.2f,%.2f)",
                    (double)wp.x, (double)wp.y);
            } else {
                cpxPrintToConsole(LOG_TO_CRTP, "frontier: none");
                wp.valid = 0;
            }
            send_waypoint(&wp);
        }

        scan_count++;
    }
}

int main(void)
{
    return pmsis_kickoff((void *)start_example);
}
```

Listing 6: map.c

```c
#include "map.h"
#include <math.h>
#include <string.h>

void map_init(OccupancyMap_t *map)
{
    memset(map->cells, 0, sizeof(map->cells));
    memset(dirty, 0, MAP_SIZE);
}

static int world_to_grid(float world)
{
    return (int)(MAP_ORIGIN + world / MAP_RESOLUTION);
}

// Cast a raw you bresham from (x0,y0) to (x1,y1). If hit is 1, mark
   the end cell as occupied, otherwise mark it as free.
static void cast_ray(OccupancyMap_t *map,
                     int x0, int y0,
                     int x1, int y1,
                     int hit)
{
    int dx =  abs(x1 - x0);
    int dy = -abs(y1 - y0);
    int sx = x0 < x1 ? 1 : -1;
    int sy = y0 < y1 ? 1 : -1;
    int err = dx + dy;

    while (1) {
        if (x0 < 0 || x0 >= MAP_SIZE || y0 < 0 || y0 >= MAP_SIZE)
            break;

        int at_end = (x0 == x1 && y0 == y1);

        if (at_end && hit) {
            int v = map->cells[y0][x0] + CELL_HIT;
            map->cells[y0][x0] = v > CELL_MAX ? CELL_MAX : (int8_t)v;
            dirty[y0] = 1;
        } else if (!at_end) {
            int v = map->cells[y0][x0] + CELL_FREE;
            map->cells[y0][x0] = v < CELL_MIN ? CELL_MIN : (int8_t)v;
            dirty[y0] = 1;
        }

        if (at_end) break;

        int e2 = 2 * err;
```

```
        if (e2 >= dy) { err += dy; x0 += sx; }
        if (e2 <= dx) { err += dx; y0 += sy; }
    }
}

void map_update(OccupancyMap_t *map,
                float pose_x, float pose_y, float pose_yaw,
                float *pts_x, float *pts_y, float *pts_z,
                int num_points)
{
    int gx0 = world_to_grid(pose_x);
    int gy0 = world_to_grid(pose_y);
    float cos_yaw = cosf(pose_yaw);
    float sin_yaw = sinf(pose_yaw);

    for (int i = 0; i < num_points; i++) {
        if (pts_z[i] > 0.01f) continue;

        float wx = pose_x + pts_x[i] * cos_yaw - pts_y[i] * sin_yaw;
        float wy = pose_y + pts_x[i] * sin_yaw + pts_y[i] * cos_yaw;

        int gx1 = world_to_grid(wx);
        int gy1 = world_to_grid(wy);

        cast_ray(map, gx0, gy0, gx1, gy1, 1);
    }
}
```

Listing 7: map.h

```c
#pragma once
#include <stdint.h>

#define MAP_SIZE       200
#define MAP_RESOLUTION 0.05f
#define MAP_ORIGIN     100

#define CELL_MAX       100
#define CELL_MIN      -100
#define CELL_HIT       10
#define CELL_FREE      -3

typedef struct {
    int8_t cells[MAP_SIZE][MAP_SIZE];
} OccupancyMap_t;

// Dirty row tracking - set by cast_ray, cleared by send_map_delta
extern uint8_t dirty[MAP_SIZE];

void map_init(OccupancyMap_t *map);

void map_update(OccupancyMap_t *map,
                float pose_x, float pose_y, float pose_yaw,
                float *pts_x, float *pts_y, float *pts_z,
                int num_points);
```

C ROS2 Client Code

Listing 8: mapper_node.py

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
from geometry_msgs.msg import PoseStamped
from std_msgs.msg import Header
import tf2_ros
from geometry_msgs.msg import TransformStamped
import numpy as np
import socket
import struct
import threading
import math

# config
AI_DECK_IP     = "192.168.4.1"
AI_DECK_PORT   = 5000
MY_PORT        = 5001
MAP_SIZE       = 200
MAP_RES        = 0.05          # metres per cell
MAP_ORIGIN     = 100           # grid cell that maps to world (0,0)
CPX_F_MAP      = 10
CPX_F_POSE     = 11
SENTINEL       = 255
#

class CfMapperNode(Node):

    def __init__(self):
        super().__init__('cf_mapper')

        # publishers
        self.map_pub  = self.create_publisher(OccupancyGrid, '/cf/map', 10)
        self.pose_pub = self.create_publisher(PoseStamped,   '/cf/pose', 10)
        self.tf_br    = tf2_ros.TransformBroadcaster(self)

        # grid state - store your raw log-odds values
        self.grid = np.zeros((MAP_SIZE, MAP_SIZE), dtype=np.int8)
        self.pose = (0.0, 0.0, 0.0)  # x, y, yaw

        # UDP socket
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", MY_PORT))
```

```
        self.sock.settimeout(3.0)

        # register with ESP32
        self.sock.sendto(b"FER", (AI_DECK_IP, AI_DECK_PORT))
        self.get_logger().info("Sent FER - waiting for map data...")

        # UDP runs in background thread - ROS2 spin runs in main thread
        self.udp_thread = threading.Thread(target=self._udp_loop,
            daemon=True)
        self.udp_thread.start()

    # UDP receive loop

    def _udp_loop(self):
        while rclpy.ok():
            try:
                data, _ = self.sock.recvfrom(4096)
                # print("test")
            except socket.timeout:
                self.get_logger().warn("UDP timeout - resending FER")
                self.sock.sendto(b"FER", (AI_DECK_IP, AI_DECK_PORT))
                continue

            if len(data) < 4:
                continue

            fn      = data[3]       # CPX function ID byte
            payload = data[4:]      # everything after CPX header

            if fn == CPX_F_MAP:
                self._handle_map_packet(payload)

            elif fn == CPX_F_POSE:
                self._handle_pose_packet(payload)

    def _handle_map_packet(self, payload):
        if len(payload) < 3:
            return

        # chunk layout: uint16 chunk_index, uint8 row, int8 data[200]
        row = payload[2]

        if row == SENTINEL:
            # delta complete - publish the map
            self._publish_map()
            return

        if len(payload) < 3 + MAP_SIZE:
```

```
            return

        # unpack 200 signed bytes starting at offset 3
        row_data = struct.unpack_from(f"<{MAP_SIZE}b", payload, 3)
        self.grid[row, :] = row_data

    def _handle_pose_packet(self, payload):
        if len(payload) < 28:
            return

        # 7 floats: pose_x, pose_y, pose_yaw, dist_f, dist_b, dist_l,
            dist_r
        vals = struct.unpack_from("<7f", payload, 0)
        self.pose = (vals[0], vals[1], vals[2])
        self._publish_pose()

    # ROS2 publishers

    def _publish_map(self):
        msg              = OccupancyGrid()
        msg.header       = Header()
        msg.header.stamp     = self.get_clock().now().to_msg()
        msg.header.frame_id = "map"

        msg.info.resolution = MAP_RES
        msg.info.width      = MAP_SIZE
        msg.info.height     = MAP_SIZE

        # world position of cell (0,0) - bottom-left corner of grid
        msg.info.origin.position.x = -MAP_ORIGIN * MAP_RES    # -5.0m
        msg.info.origin.position.y = -MAP_ORIGIN * MAP_RES    # -5.0m
        msg.info.origin.position.z = 0.0
        msg.info.origin.orientation.w = 1.0

        # remap log-odds → ROS2 convention
        # ROS2: -1 = unknown, 0 = free, 100 = occupied
        ros_grid = np.full((MAP_SIZE, MAP_SIZE), -1, dtype=np.int8)
        ros_grid[self.grid < 0]  = 0      # free cells
        ros_grid[self.grid > 0]  = 100    # occupied cells
        # cells == 0 stay as -1 (unknown)

        msg.data = ros_grid.flatten().tolist()
        self.map_pub.publish(msg)

    def _publish_pose(self):
        px, py, pyaw = self.pose
        now = self.get_clock().now().to_msg()

```

```
        # PoseStamped for RViz2 arrow marker
        pose_msg = PoseStamped()
        pose_msg.header.stamp    = now
        pose_msg.header.frame_id = "map"
        pose_msg.pose.position.x = px
        pose_msg.pose.position.y = py
        pose_msg.pose.position.z = 0.0
        pose_msg.pose.orientation.z = math.sin(pyaw / 2.0)
        pose_msg.pose.orientation.w = math.cos(pyaw / 2.0)
        self.pose_pub.publish(pose_msg)

        # TF transform - lets RViz2 show the drone frame axes
        t = TransformStamped()
        t.header.stamp    = now
        t.header.frame_id = "map"
        t.child_frame_id  = "drone"
        t.transform.translation.x = px
        t.transform.translation.y = py
        t.transform.translation.z = 0.0
        t.transform.rotation.z = math.sin(pyaw / 2.0)
        t.transform.rotation.w = math.cos(pyaw / 2.0)
        self.tf_br.sendTransform(t)


def main():
    rclpy.init()
    node = CfMapperNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```